

Justice Case Management System

With Next.js, Node.js, TiDB, and Blockchain

Team Presentation

This presentation is divided by technology so that each team member explains one clear part of the system.

Next.js | Node.js | TiDB | Blockchain

Problem Statement

Manual or weakly protected justice case systems make it difficult to manage records, track workflow changes, and prove that sensitive case data has not been tampered with.

Operational challenge: records move through many workflow stages.

Integrity challenge: important changes must be traceable and verifiable.

SMART Objective

To build a justice case management system that allows users to capture, process, store, and verify case records using Next.js, Node.js, TiDB, and blockchain, with traceable workflow actions and integrity checks during the project demonstration period.

Best Team Order

1. Next.js
2. Node.js
3. TiDB
4. Blockchain

Reason: user interface -> backend logic -> database storage -> integrity verification

Combined line: Next.js handles the interface, Node.js handles the logic, TiDB stores the full data, and blockchain provides proof of integrity.

User 1 - Next.js

Part: Frontend pages, API routes, user interaction, form submission flow.

Files:

- app/police/new-case/page.tsx
- app/api/auth/login/route.ts
- app/api/cases/route.ts
- app/api/blockchain/anchor/route.ts

Functions: buildCasePayload(), POST() login, GET()/POST() cases, POST() blockchain an

Say: My part is Next.js. It handles the user interface and the API routes.

User 2 - Node.js

Part: Backend logic, authentication handling, workflow processing, case management.

Files:

- lib/server/cases.ts
- lib/server/auth-session.ts
- lib/server/users.ts

Functions: parseSessionValue(), getSessionUserFromRequest(), listCases(), createAndSubmitCase(), checkpointPoliceCaseStep().

Say: My part is Node.js. It handles the server-side logic behind the scenes.

User 3 - TiDB

Part: Database storage, secure connection, structured records, SQL management.

Files:

- lib/db.ts
- db/offchain.sql
- lib/db/tidb.ts

Functions: getDbPool(), readCalfProvided(), toBool().

Strong line: TiDB stores the full business record.

User 4 - Blockchain

Part: Integrity verification, hash generation, smart contract interaction, tamper evidence.

Files:

- lib/blockchain/case-anchor.ts
- chain/contracts/CaseAnchor.sol
- app/api/blockchain/anchor/route.ts
- lib/blockchain.ts

Functions: getContract(), makeCaseHash(), anchorCaseOnChain().

Strong line: Blockchain stores proof of integrity and critical action history.

Closing

Final important sentence:

The database stores the full business record, while blockchain stores proof of integrity and critical action history.

Short closing:

This system combines usability, backend control, structured storage, and tamper evidence in one platform.